

Taller Práctico: Asincronía y Callbacks en JavaScript.

Objetivo.

Implementar y analizar soluciones utilizando callbacks y tareas asíncronas en JavaScript para comprender cómo funciona el comportamiento no bloqueante del lenguaje.

—

Parte 1. Callbacks Básicos.

Un callback es una función que se envía como argumento a otra función para ser ejecutada posteriormente.

Analiza el siguiente código y completa las secciones marcadas con TODO.

```
function realizarOperacion(num1, num2, operacionCallback) {
    console.log(`Operación con: ${num1} y ${num2}`);
    operacionCallback(num1, num2);
}

function sumar(a, b) {
    console.log(`Resultado Suma: ${a + b}`);
}

// TODO
function restar(a, b) {

}

// TODO
function multiplicar(a, b) {

}

realizarOperacion(10, 5, sumar);

// TODO
// Implementar las tres operaciones.
```

Actividades.

1. Implementa el callback `restar`.
2. Implementa el callback `multiplicar`.
3. Ejecuta las tres operaciones.
4. Captura la salida obtenida en consola.

Preguntas de reflexión.

1. ¿Qué ventaja tiene enviar una función como parámetro?

Permite reutilizar una misma función para ejecutar diferentes tareas sin modificar su código.

2. ¿Qué ocurre si enviamos una función diferente como callback?

Se ejecutará la nueva función enviada como parámetro.

3. ¿La función principal necesita conocer qué operación matemática se ejecutará?

No. Solo necesita llamar al callback recibido.

—

Parte 2. Comprendiendo la Asincronía.

Observa cuidadosamente el siguiente programa.

```
function tareaNoBloqueante(callback) {
    console.log("Iniciando tarea no bloqueante...");

    setTimeout(function() {
        console.log("Tarea completada.");
        callback();
    }, 2000);
}

console.log("Inicio del programa.");

tareaNoBloqueante(function() {
    console.log("Continuando después de la tarea.");
});

console.log("Fin del programa.");
```

Actividad.

Antes de ejecutar el programa:

1. Escribe el orden que crees que aparecerá en la consola.

Inicio del programa, iniciando tarea no bloqueante, fin del programa, tarea completada, continuando después de la tarea.

2. Ejecuta el código.

Inicio del programa, iniciando tarea no bloqueante, fin del programa, tarea completada, continuando después de la tarea.

3. Registra el resultado real.

Inicio del programa, iniciando tarea no bloqueante, fin del programa, tarea completada, continuando después de la tarea.

Análisis.

Responde:

1. ¿Coincidió tu predicción con el resultado?

Sí, la predicción coincide con el resultado obtenido.

2. ¿Por qué el mensaje "Fin del programa" aparece antes que "Tarea completada"?

Porque `setTimeout()` programa la ejecución para más tarde y el resto del código continúa ejecutándose inmediatamente.

3. ¿Qué papel cumple `setTimeout()` en este comportamiento?

`setTimeout()` permite ejecutar una función después de un tiempo determinado sin detener la ejecución del programa.

—

Parte 3. Simulación de Procesos Reales.

Debes simular tres tareas comunes de una aplicación moderna.

Código Base.

```
function solicitarJSON(callback) {  
  
}  
  
function reproducirAudio(callback) {  
  
}  
  
function leerSensor(callback) {  
  
}  
  
console.log("--- Iniciando procesos asíncronos ---");  
  
solicitarJSON(() => console.log("Archivo JSON recibido."));  
reproducirAudio(() => console.log("Audio finalizado."));  
leerSensor(() => console.log("Datos del sensor listos."));
```

Requisitos.

Implementa cada función utilizando `setTimeout()`.

Función.	Tiempo.
solicitarJSON	3000 ms
reproducirAudio	1000 ms
leerSensor	500 ms

Actividades.

1. Implementa las tres funciones.
2. Ejecuta el programa.
3. Registra el orden real de finalización.

Resultado.

Orden	Tarea
1	<code>leerSensor()</code>
2	<code>reproducirAudio()</code>
3	<code>solicitarJSON()</code>

Preguntas.

1. ¿Cuál terminó primero?

`leerSensor()` porque tiene un tiempo de espera de 500 ms.

2. ¿Cuál terminó último?

`solicitarJSON()` porque tiene un tiempo de espera de 3000 ms.

3. ¿Por qué el orden de finalización es diferente al orden de ejecución?

Porque las tareas se ejecutan de manera asíncrona y cada una tiene un tiempo de espera distinto. La que tiene menor tiempo termina primero aunque haya sido llamada después.

Parte 4. Diseñando tu Propia Operación Asíncrona.

Reto.

Crea una función llamada:

`simularDescarga(nombreArchivo, callback)`

La función deberá:

1. Mostrar un mensaje indicando que inició la descarga.
2. Esperar 4 segundos utilizando `setTimeout()`.
3. Mostrar un mensaje indicando que la descarga terminó.
4. Ejecutar el callback recibido.

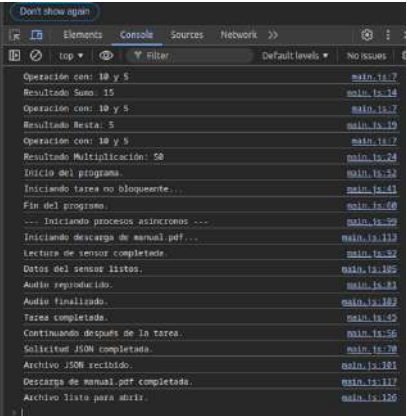
Ejemplo esperado.

```
simularDescarga("manual.pdf", () => {  
    console.log("Archivo listo para abrir.");  
});
```

Evidencia.

Captura la salida de consola y explica qué sucede en cada paso.

Taller Práctico: Asincronía y Callbacks en JavaScript.



Parte 5. Análisis del Callback Hell.

Observa la siguiente estructura:

```
obtenerUsuario(() => {  
  obtenerPedidos(() => {  
    obtenerFactura(() => {  
      enviarCorreo(() => {  
        console.log("Proceso finalizado");  
      });  
    });  
  });  
});
```

Actividades.

1. Describe el problema visual que presenta el código.

El código presenta una estructura muy anidada que se desplaza cada vez más hacia la derecha. Esto hace que sea difícil de leer y comprender.

2. Explica qué es el Callback Hell.

El Callback Hell es una situación que ocurre cuando varios callbacks se anidan unos dentro de otros para ejecutar tareas asíncronas de forma secuencial.

A medida que aumentan los niveles de anidación, el código se vuelve más complejo, menos legible y más difícil de mantener.

3. ¿Por qué dificulta el mantenimiento de aplicaciones grandes?

Porque:

4. **Reduce la legibilidad del código.**
5. **Hace más difícil encontrar errores.**
6. **Complica la depuración.**
7. **Aumenta la dificultad para agregar nuevas funcionalidades.**
8. **Genera una estructura desordenada conocida como "pirámide de la muerte" (Pyramid of Doom).**

4. Investiga y menciona dos alternativas modernas para evitar este problema.

Alternativa 1: Promesas (Promises)

Las Promesas permiten encadenar operaciones asíncronas utilizando `.then()`, evitando múltiples niveles de anidación.

Ejemplo:

obtenerUsuario()

`.then(obtenerPedidos)`

`.then(obtenerFactura)`

`.then(enviarCorreo)`


```
.then() => {  
    console.log("Proceso finalizado");  
};
```

async y **await** permiten escribir código asíncrono con una sintaxis similar al código síncrono, facilitando su lectura y mantenimiento.

Ejemplo:

```
async function proceso() {  
  
    await obtenerUsuario();  
  
    await obtenerPedidos();  
  
    await obtenerFactura();  
  
    await enviarCorreo();  
  
    console.log("Proceso finalizado");  
}  
  
proceso();
```

Entregables.

El estudiante deberá entregar:

- Código fuente de todos los ejercicios.
- Capturas de consola de las ejecuciones.
- Respuestas a las preguntas de análisis.
- Explicación del ejercicio de Callback Hell.